Version 1 Project Review

Project Overview

My original goal was to build both a Steam review scraper and an interactive application for analyzing Steam reviews. As development progressed and with conversations with Prof Harding, I realized that trying to build both a front-end interface and a comprehensive analysis tool would spread the project too thin. After evaluating the project's direction, we decided to narrow the scope and focus on creating a robust backend analysis pipeline.

The project is now centered around collecting, processing, categorizing, visualizing, and statistically analyzing Steam review data. By reducing the project scope, I was able to spend more time implementing meaningful statistical analyses, improving the code structure, and creating a reusable analysis workflow rather than building interface components.

What Has Been Implemented So Far

Data Processing

- Load Steam review data from CSV files.
- Remove duplicate reviews.
- Clean and prepare the dataset for analysis.

Natural Language Processing

- Implement sentiment analysis using TextBlob.
- Categorize reviews into gameplay-related themes using keyword matching.

Feature Engineering

- Calculate review length.
- Calculate word count.
- Calculate sentiment strength.
- Generate True/False (Boolean) category columns to support statistical analyses.

Statistical Analysis

- Descriptive statistics.
- Histograms.
- Category frequency bar charts.
- Box plots.
- Correlation matrices.
- T-tests.
- Chi-square tests.
- Linear regression analysis.

AI-Assisted Interpretation

- Integrated Ollama to automatically generate summaries of statistical analyses and visualizations, making the results easier to interpret and reducing manual explanation.

Demonstration of Current Functionality

The notebook now performs the following workflow:

1. Loads and cleans Steam review data.
2. Performs sentiment analysis.
3. Categorizes each review into one or more gameplay-related topics.
4. Added in addtional features such as review length and sentiment strength.
5. Generates descriptive statistics and visualizations.
6. Runs statistical analyses, including correlations, t-tests, chi-square tests, and regression.
7. Installed Ollama to automatically generate AI summaries explaining each visualization and statistical result.

Issues Encountered and Solutions

1. Notebook Organization Became Difficult

As I continued adding new analyses, the notebook became increasingly difficult to manage because many cells depended on previously executed cells. Debugging became more time-consuming, and it was harder to identify where errors originated.

Solution

Rather than continuing to build on a notebook that gets longer and longer, I created a new notebook and reorganized the project into clearly defined sections. Helper functions, feature engineering, statistical analyses, and the final analysis pipeline are now separated into logical modules, making the notebook easier to understand, debug, and extend.

---

2. Repetitive Statistical Code

Initially, every visualization and statistical analysis required writing similar plotting and interpretation code repeatedly.

Solution

I created reusable analysis functions that automatically:

- Generate the visualization.
- Perform the statistical analysis.
- Send the results to Ollama.
- Display an AI-generated interpretation.

This significantly reduced duplicated code and made it much easier to add new analyses.

---

3. Package Installation and Dependency Issues

During development, I encountered several issues involving package installation and environment configuration, including SciPy, TextBlob, Statsmodels, and Ollama. I also experienced problems with notebook kernel restarts and missing dependencies.

Solution

I standardized the notebook setup, documented required packages, and resolved dependency issues so the notebook can be run from a clean environment with minimal setup.

---

4. Regression Analysis Errors

When implementing regression analysis, Statsmodels initially produced errors because some variables were being interpreted as object data types instead of numeric values.

Solution

I converted the appropriate columns to numeric values and cleaned missing data before fitting the regression model.

Remaining Challenges

Although the core functionality is now working, there are still several areas I would like to improve:

- Expand the keyword dictionaries used for categorization.
- Improve AI-generated summaries to provide deeper statistical insight.
- Add additional statistical analyses and effect size calculations.
- Continue refining the documentation and presentation of results.

Milestones for Version 2

Milestone 1: Expand Review Categories

Improve and expand the keyword dictionaries to better capture common themes found in Steam reviews.

Milestone 2: Advanced Statistical Analysis

Add additional statistical techniques such as effect sizes, confidence intervals, and more advanced comparisons where appropriate.

Milestone 3: Improve AI Interpretation

Refine the Ollama prompts to produce more detailed and insightful summaries of statistical results.

Milestone 4: Final Project Documentation

Complete the README, GitHub documentation, final report, and presentation materials.

Self Reflection

Overall, I think my progress is on the right track. Although I reduced the original project scope by removing the front-end application component, I believe this decision created a stronger and more technically focused project. The project has evolved from being primarily a data collection tool into a reusable data analysis pipeline that combines natural language processing, statistical analysis, data visualization, and AI-assisted interpretation.

I think the core functionality is now complete. The remaining work will focus primarily on expanding the analyses, refining the AI summaries, improving documentation, and polishing the final presentation.

I am confident that I can complete the project within the remaining course timeline.

Expectations vs. Reality

Prof Hardings help with loading the CSV data, performing basic sentiment analysis with TextBlob, and implementing keyword-based review categorization was very helpful in kicking off this project.

Other aspects proved more challenging than anticipated. Organizing a growing notebook, managing package dependencies, implementing statistical models, debugging execution order within Jupyter notebooks, and configuring Ollama required considerably more time than I had expected.

Despite these challenges, working through each issue improved both the quality of the project and my understanding of data analysis workflows.

Light Bulb Moments

One of the biggest realizations during this project was that condensing the functions and making them reusable reduced duplicated code. Instead of writing separate plotting and statistical code for every analysis, I could write generalized functions that accepted different variables as parameters. This condensed the notebook and made it easier to understand.

Another item that I realized was that AI could contribute more than just text analysis. Initially, I expected AI to help analyze the review of the data output itself. As I continued to run the code, I realized it could also improve the usability of the project by automatically interpreting statistical outputs and visualizations. Integrating Ollama transformed AI into an assistant that explains analytical results rather than simply processing data.

Finally, I learned that reorganizing existing code can be just as valuable as adding new functionality. Rebuilding the notebook with a clear structure, which made debugging easier, improved readability, and made future coding much easier.